Université Mohamed V

École Nationale Supérieure d'Arts et Métiers

Rabat

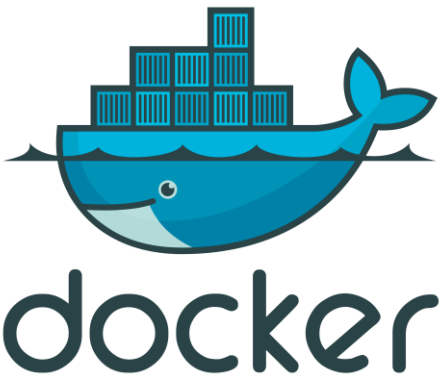
Année Universitaire : 2026/27

Filière : INDIA

TP3 : PROJET ML COMPLET

ANALYSE DE SENTIMENT YOUTUBE

Du Développement au Déploiement Cloud



Date : 04/11/2026

Prof. Maria Zemzami

CONTEXTE DU PROJET

PROBLEMATIQUE

Les créateurs de contenu YouTube reçoivent des centaines, voire des milliers de commentaires sur leurs vidéos. L'analyse manuelle de ces commentaires pour comprendre le sentiment général de l'audience est :

- Chronophage et peu scalable
- Subjective et biaisée
- Difficile à quantifier pour des décisions stratégiques

VOTRE MISSION

Développer un système MLOps complet qui permet d'analyser automatiquement le sentiment des commentaires YouTube en temps réel, avec :

- Un modèle ML entraîné et optimisé
- Une API REST déployée dans le cloud
- Une extension Chrome pour l'interface utilisateur
- Un pipeline de tests et validation

ARCHITECTURE DU SYSTEME

Votre système devra implémenter l'architecture suivante :

Composant	Description
Frontend	Extension Chrome (JavaScript, HTML, CSS)
Backend	API REST FastAPI avec endpoints de prédiction
ML Engine	Modèle scikit-learn (TF-IDF + Logistic Regression)
Déploiement	Hugging Face Spaces (Docker)
Version Control	Git + GitHub

FLUX DE DONNEES

Étape 1: L'utilisateur visite une vidéo YouTube
Étape 2: L'extension Chrome extrait les commentaires de la page
Étape 3: Les commentaires sont envoyés par batch à l'API cloud
Étape 4: L'API traite les données avec le modèle ML
Étape 5: Les prédictions de sentiment sont retournées
Étape 6: L'extension affiche les résultats avec visualisations

PREREQUIS

COMPTES REQUIS

- **Compte Hugging Face** pour héberger l'API
- **Compte GitHub** pour le versioning

LOGICIELS NECESSAIRES

- Python 3.10 ou supérieur
- Git (pour le versioning)
- Google Chrome (pour tester l'extension)
- Un éditeur de code

PHASES DU PROJET

PHASE 1 : CONFIGURATION DE L'ENVIRONNEMENT MLOPS

Objectif : Mettre en place une structure de projet

TACHES A REALISER

- Créer une structure de dossiers MLOps
- Initialiser un repository Git
- Créer et activer un environnement virtuel Python
- Définir les dépendances dans requirements.txt
- Configurer un .gitignore approprié

Structure de dossiers suggérée :

```
data/ (raw, processed)

models/ (modèles entraînés)

src/ (data, models, api, utils)

chrome-extension/ (frontend)

logs/ (fichiers de log)
```

Questions de réflexion :

- Pourquoi séparer les données raw et processed ?
- Quels fichiers ne doivent jamais être commités sur Git ?
- Quelle est l'importance d'un environnement virtuel ?

PHASE 2 : PIPELINE DE DONNEES ET PREPARATION

Objectif : Collecter et préparer les données d'entraînement

TACHES A REALISER

- Télécharger le dataset de commentaires labelisés

  Dataset à utiliser :

  Vous devez télécharger le dataset Reddit Sentiment Analysis disponible à l'adresse suivante :

  https://raw.githubusercontent.com/Himanshu-1703/reddit-sentiment-analysis/refs/heads/main/data/reddit.csv

  Ce dataset contient des commentaires Reddit labelisés avec les catégories de sentiment suivantes :

- -1 : Négatif

- 0 : Neutre

- 1 : Positif

Votre script de téléchargement doit :

- Télécharger automatiquement le fichier CSV depuis cette URL

- Sauvegarder les données dans data/raw/reddit.csv

- Afficher des statistiques sur le dataset (nombre de commentaires, distribution des labels)

- Créer un script de nettoyage des données (suppression des URLs, mentions, caractères spéciaux)
- Implémenter une analyse exploratoire (distribution des classes, longueur des textes)
- Gérer le déséquilibre des classes si nécessaire
- Créer un split train/test reproductible

Spécifications techniques :

Labels de sentiment	-1 (négatif), 0 (neutre), 1 (positif)
Taille minimale du dataset	1000+ commentaires (au moins 300 par classe)
Format de stockage	CSV avec colonnes 'text' et 'label'

Questions de réflexion :

- Pourquoi nettoyer les URLs et mentions ?
- Comment gérer les emojis dans le texte ?
- Quel ratio train/test est approprié ?

PHASE 3 : DEVELOPPEMENT ET ENTRAINEMENT DU MODELE

Objectif : Créer un modèle de classification de sentiment performant

TACHES A REALISER

- Implémenter un vectoriseur TF-IDF avec paramètres optimisés
- Entraîner un modèle de Logistic Regression
- Expérimenter avec d'autres algorithmes (Random Forest, SVM, etc.)
- Optimiser les hyperparamètres avec GridSearchCV ou RandomizedSearchCV
- Évaluer avec métriques appropriées (accuracy, F1-score, matrice de confusion)
- Sauvegarder le meilleur modèle et le vectoriseur avec joblib

Critères de performance :

- Accuracy minimale : 80%
- F1-score par classe > 0.75
- Temps d'inférence < 100ms pour un batch de 50 commentaires

Questions de réflexion :

- Pourquoi utiliser TF-IDF plutôt que Count Vectorizer ?
- Comment interpréter la matrice de confusion ?
- Quels hyperparamètres ont le plus d'impact sur les performances ?

PHASE 4 : DEVELOPPEMENT DE L'API REST

Objectif : Créer une API FastAPI pour servir le modèle

TACHES A REALISER

- Créer une application FastAPI avec structure modulaire
- Implémenter un endpoint /health pour le monitoring
- Créer un endpoint /predict_batch pour traiter plusieurs commentaires
- Ajouter la validation de données avec Pydantic
- Configurer CORS pour autoriser les requêtes depuis l'extension Chrome
- Implémenter la gestion d'erreurs appropriée
- Tester localement avec uvicorn

Spécifications des endpoints :

Endpoint	Méthode	Description
`/health`	GET	Vérifie l'état de l'API et du modèle
`/predict_batch`	POST	Analyse un batch de commentaires et retourne sentiments + statistiques

Questions de réflexion :

- Pourquoi utiliser FastAPI plutôt que Flask ?
- À quoi sert la configuration CORS ?
- Comment gérer efficacement le chargement du modèle ?

PHASE 5 : DEVELOPPEMENT DE L'EXTENSION CHROME

Objectif : Créer une interface utilisateur professionnelle

TACHES A REALISER

- Créer le fichier manifest.json avec les permissions appropriées
- Développer le content script pour extraire les commentaires YouTube
- Créer l'interface popup avec HTML/CSS moderne
- Implémenter la logique JavaScript pour communiquer avec l'API
- Ajouter des visualisations (graphiques, statistiques)
- Implémenter des fonctionnalités : filtres, dark mode, export
- Gérer les états de chargement et les erreurs

Fonctionnalités requises :

- Extraction automatique des commentaires visibles
- Affichage des statistiques globales (%, répartition)
- Liste des commentaires avec leur sentiment et confiance
- Filtres par sentiment (positif/neutre/négatif)
- Mode sombre/clair
- Fonction copier les résultats

Questions de réflexion :

- Comment gérer les commentaires avec des réponses (threads) ?
- Quelle stratégie pour extraire les commentaires dynamiques ?
- Comment optimiser les performances avec de nombreux commentaires ?

PHASE 6 : DEPLOIEMENT CLOUD AVEC DOCKER

Objectif : Déployer l'API sur Hugging Face Spaces

TACHES A REALISER

- Créer un fichier app_api.py optimisé pour le déploiement
- Écrire un Dockerfile avec les bonnes pratiques
- Créer un requirements.txt de production (uniquement les dépendances nécessaires)
- Créer un Space sur Hugging Face avec SDK Docker
- Pousser le code et les modèles vers Hugging Face
- Vérifier le déploiement et tester les endpoints
- Mettre à jour l'extension Chrome avec l'URL de production

Bonnes pratiques Docker :

- Utiliser une image de base légère (python:3.10-slim)
- Copier uniquement les fichiers nécessaires
- Exposer le port 7860 (standard Hugging Face)
- Optimiser les layers pour le cache

Questions de réflexion :

- Pourquoi séparer les requirements dev et prod ?
- Comment réduire la taille de l'image Docker ?
- Quelle stratégie pour les mises à jour du modèle ?

PHASE 7 : TESTS ET VALIDATION

Objectif : Assurer la qualité et la fiabilité du système

TESTS A EFFECTUER

TESTS DU MODELE

- Vérifier la performance sur le test set
- Tester avec des cas limites (texte très court/long, emojis, langues mélangées)
- Mesurer le temps d'inférence

TESTS DE L'API

- Tester le endpoint /health
- Tester /predict_batch avec différentes tailles de batch
- Vérifier la gestion d'erreurs (données invalides, batch vide)
- Mesurer le temps de réponse

TESTS DE L'EXTENSION

- Installation et chargement dans Chrome
- Extraction des commentaires sur différentes vidéos
- Toutes les fonctionnalités (filtres, dark mode, copie)
- Gestion des erreurs réseau

TESTS D'INTEGRATION

- Test du flux complet : Extension → API Cloud → Résultats

- Test avec API locale puis avec API déployée
- Vérifier la cohérence des résultats

LIVRABLES

Votre projet doit inclure les éléments suivants :

1. CODE SOURCE

- **Repository GitHub** avec structure MLOps complète
- Scripts de traitement des données
- Code d'entraînement du modèle
- Application FastAPI
- Extension Chrome complète
- Dockerfile et fichiers de configuration

2. MODELE ENTRAINE

- Fichiers .joblib (modèle + vectoriseur)
- Rapport de performance (métriques, visualisations)

3. API DEPLOYEE

- **URL Hugging Face Space** fonctionnelle
- Documentation API (endpoints, exemples de requêtes)

4. EXTENSION CHROME

- Extension packagée (.zip ou dossier)
- Instructions d'installation
- Guide utilisateur avec captures d'écran

5. DOCUMENTATION

- **README.md** complet sur GitHub avec :

- Description du projet

- Architecture technique

- Instructions d'installation et d'utilisation

- Démonstration avec screenshots

- **Rapport technique PDF** (8-12 pages) incluant :

- Analyse du problème et solution proposée

- Architecture détaillée avec diagrammes

- Choix techniques justifiés

- Résultats et métriques de performance

- Difficultés rencontrées et solutions

- Améliorations futures possibles

6. VIDEO DE DEMONSTRATION

- Vidéo de 5-8 minutes montrant :

- Installation de l'extension
- Utilisation sur une vraie vidéo YouTube
- Toutes les fonctionnalités (filtres, dark mode, etc.)
- Appel à l'API et temps de réponse

RESSOURCES ET REFERENCES

DOCUMENTATION OFFICIELLE

- FastAPI : https://fastapi.tiangolo.com/
- Scikit-learn : https://scikit-learn.org/
- Hugging Face Spaces : https://huggingface.co/docs/hub/spaces
- Chrome Extensions : https://developer.chrome.com/docs/extensions/
- Docker : https://docs.docker.com/

CONCEPTS MLOPS ESSENTIELS

- Séparation data/code/models pour la reproductibilité
- Versioning avec Git (commits atomiques, messages clairs)
- Containerisation pour le déploiement reproductible
- API REST pour servir les modèles ML
- Monitoring et logging en production

Bon travail !